HTML-CSS-03 第3回_まとめ

第3回の目標

- **レスポンシブの書き方を知る**
- **レスポンシブ化させる流れをイメージできるようになる**
- **headタグ内に書く情報を知る**

レスポンシブに必要な材料3つ🥕

① ビューポート

レスポンシブを導入するためにHTML側に必要なもの（基本はテンプレートに書いてある）。

```
<meta name="viewport" content="width=device-width, initial-scale=1.0" />
```

- `width=device-width`：表示領域の幅を端末の画面幅に合わせる
- `initial-scale=1.0`：ページを開いたときの拡大率を等倍にする
- 📎 viewportの詳細

② メディアクエリ

画面幅が違うサイズになったときのスタイルを書くエリア。

```
@media screen and (min-width: 〇〇px) {
  .class {
    color: #fff;
  }
}
```

❗もともと書いていたCSSの必ず**下**に書く❗

書き方	意味
`min-width`	指定した幅**以上**で適用（最低値を決める）
`max-width`	指定した幅**以下**で適用（上限を決める）

- どちらを使うかで意味が変わるので注意。モバイルファーストなので`min-width`を使う

【＋α】メディアクエリは入れ子でも書ける

最新の書き方では、クラスの中にメディアクエリを書くこともできる。

```
.box {
  color: blue;

  /* 800px以上のときだけ赤にする */
  @media (min-width: 800px) {
    color: red;
  }
}
```

- Sassを使わないCSSネスティング

メディアクエリをスニペット登録する

`@sm`・`@lg`と打つだけでメディアクエリが出るようにする。 1. VSCode左下の歯車アイコンを選択 2. 「ユーザースニペットの構成」をクリック 3. 上の検索バーで「css」と検索 4. 「css.json」（または「css」）をクリック 5. もともとあるコードを消して、下のコードを貼り付ける

```
{
    "media query sm": {
        "prefix": "@sm",
        "body": ["@media screen and (min-width: 768$2px) {", "  $1", "}"],
        "description": "media query for css"
    },
    "media query lg": {
        "prefix": "@lg",
        "body": ["@media screen and (min-width: 1080$2px) {", "  $1", "}"],
        "description": "media query for css"
    }
}
```

- `prefix`：呼び出すときに打つ文字
- `body`：展開されるコード。`$1`・`$2`はカーソルが移動する位置
  - `$1`：Enterを押したあと、最初にカーソルが当たる位置
  - `$2`：Tabキーを押したあとにカーソルが当たる位置（3以降も使える）
- VSCodeに最初から入っているスニペットもある（HTMLで`!`と打つと雛形、JSで`log`と打つと`console.log()`）

- ユーザースニペット（VSCode公式）／ユーザースニペットとは／よく使うVSCodeスニペット集

【＋α】Range記法

最新のメディアクエリでは、比較演算子（`<=`、`>=`、`<`、`>`）を使ったRange（範囲）記法が主流になってきている。`min-width`・`max-width`より直感的に書ける。 - メディアクエリのRange記法／メディアクエリで比較演算子を使う

③ ブレイクポイント

レスポンシブのスタイルを適用させたい画面幅の数値。＜目安＞ - タブレット(Tab)：768px - パソコン(PC)：1080px

ブレイクポイントの決め方

- 案件や現場ごとにルールが違う。ルールがあればそれに従う
- 自分で決めるときは、ターゲット層の主流のデバイスに合わせるのが理想。ただしデバイスは多様化しているので、使いやすいブレイクポイントを決めて使うのが現実的
- 流れ
    1. メインのブレイクポイントを2〜3個決める（まずは教材通りでよい）
    2. そのブレイクポイントでレスポンシブを完成させる
    3. 画面幅を320px〜2000pxまで細かく確認し、表示崩れがないか見る
    4. 必要なところだけ、ブレイクポイントを追加したりCSSを変えたりする
- 最近の傾向は「レスポンシブ ブレイクポイント 2025」などで調べる
- レスポンシブデザインのブレイクポイントまとめ／俺流レスポンシブコーディング／レスポンシブ対応の考え方と実装

レスポンシブの作業の流れ

レスポンシブをする際は、以下の流れで作業を進めていく。 1. **デザインデータを見比べる** 2. **変更箇所を見つける** 3. **CSSでメディアクエリとスタイルを記述する**

上記の繰り返しが基本。スマホ・PCの間のサイズは、時にデザインデータにない仕様を汲み取ってコーディングを行う必要がある。

- HTMLの構造を確認・イメージしておくと、レスポンシブがうまくいきやすい
- 余白・文字の大きさは注意して確認する。もともとのスタイルに書かれているプロパティがヒントになる（値を変えるだけのことが多い）
- もともと当たっている余白と、メディアクエリで追加した余白が二重にならないか確認する。二重になるときは`padding: 0;`などで打ち消す

検証ツールで出てくる色の見方

色	意味
水色	要素
オレンジ色	margin
黄緑色	padding
紫	gap

変える順番は「レイアウト（縦並び・横並び・重なる）→ 大きさ（横幅・高さ）→ その他（余白・色・文字サイズなど)」の順で見ると調整しやすい。

よくある間違い

- メディアクエリを使わず、スマホ用の記述そのものを書き換えてしまう → スマホ表示が崩れる。スマホのスタイルはそのままにして、メディアクエリで追記する
- 上書きする必要のないプロパティまでメディアクエリの中に書く → エラーにはならないが不要なコード。上書きするものだけ書く
- メディアクエリをCSSの一番下にまとめて書く → どのクラスにかかっているか分かりにくく、元のスタイルを見るのにスクロールが必要になる。元のクラスのすぐ下に書く
- 要素を消すときに`opacity: 0;`を使う → 透明になるだけで要素は残るので、配置の邪魔になる。`display: none;`で消す
- 消したい要素の範囲を間違える（例：ハンバーガーの線だけ消して、ボタン本体が残る）→ HTMLでどこまでが対象か確認する
- CSSで要素を非表示にする3つの方法

デザインデータを確認する際の主な変更箇所

- 横並びの箇所を縦並びにする（またはその逆）
- テキストの大きさ／改行を調節する
- 要素の横幅／高さを調節する
- 要素の表示／非表示を切り替える
- 余白の大きさを調節する

教材のレスポンシブ仕様（今回の例）

- スマホ版のスタイル：cssファイルに記述済み（モバイルファーストで書かれているので書き換え不要）
- タブレット版のスタイル：画面幅が768px〜1080px未満の間に適用 `CSS @media screen and (min-width: 768px) { }`

- パソコン版のスタイル：画面幅が1080pxより大きい際に適用 `CSS @media screen and (min-width: 1080px) { }`

レスポンシブでよく使うテクニック

横並び⇄縦並びの切り替え

画面幅によって「横並びにしたい要素」が変わることはよくある。 - `display: flex;` は、横並びにしたい要素の直前の親要素にかける - メディアクエリを書く場所は、ベースのスタイルのすぐ下（「ベースのスタイル→768px→1080px」の順で書くとうまく反映される）

```
.sample {
  display: block;
}

@media screen and (min-width: 1080px) {
  .sample {
    display: flex;
    align-items: center;
    justify-content: space-between;
  }
}
```

- flexで横並びにした要素の位置調整は、まずflex系のプロパティで解決できないか考える
- `flex-direction` ：flexアイテムの並ぶ向きを決める。 `column` で縦並び、 `row` で横並び

```
/* スマホ：flexだが縦並び */
.sample_container {
  display: flex;
  align-items: center;
  flex-direction: column;
  gap: 40px;
}

/* PC：横並びに上書きする（変わる部分だけ書く） */
@media screen and (min-width: 1080px) {
  .sample_container {
    flex-direction: row;
```

```
    justify-content: center;
  }
}
```

- 横並びを解除したいときは`display: block;`をあてる
- 知らないCSSがあったら「css flex-direction」のようにそのまま調べる。AIにコードを貼って「なぜ縦並びになるのか」と聞くのもよい

値（`justify-content`）	配置
`space-between`	最初と最後の子要素を両端に置き、残りを均等な間隔で配置
`space-around`	両端の子要素も含めて、均等な間隔で配置

- Emmetは単語の頭文字で出てくることが多い（例：`jcsb` → `justify-content: space-between;`）。暗記は不要
- Flexboxプロパティ一覧
- 【図解】親要素・子要素

画像がつぶれないようにする

横並びにした画像とテキストを`flex`で並べたとき、画面幅が狭まると画像側の横幅まで縮んでしまうことがある。画像側に`flex-shrink: 0;`をつけておくと、`flex`で横並びにしたときに画像の横幅が縮まらないようにできる。 - shrink＝「縮む」。縮み率を操作するプロパティで、`0`以外はほぼ使わない - flexをかけると、入れ物の中で要素が押し合ってつぶれる（満員電車のイメージ） - 使い方のポイント 1. flexで横並びになっている要素に使う 2. 縮ませたくない要素の大きさを指定する 3. 縮ませたくない方に`flex-shrink: 0;`をつける

widthとmax-widthの違い

プロパティ	意味
`width`	要素の幅を指定した値に固定する
`max-width`	要素の幅が指定した値を超えないように制限する（画面が小さいときは追従して縮む）

```
.sample {
  width: 100%;      /* 画面幅に合わせて縮む */
  max-width: 960px; /* PCでも960pxより大きくならない */
}
```

- 可変にしつつ上限を設けたいときの組み合わせで、レスポンシブでよく使う

中央寄せ

要素の左右の`margin`を`auto`にすると、その要素を中央寄せにできる。

```
.sample {
  max-width: 872px;
  margin-right: auto;
  margin-left: auto;
}
```

- `margin`左右`auto`で中央寄せできる条件
    1. 表示形式がブロックであること
    2. `width`か`max-width`が指定されていること
- [そもそもmargin: auto;って何？]

max-widthを872pxにする理由

- 840px（記事の幅）＋ 32px（左右のpadding 16px × 2）
- `box-sizing: border-box;`だと、paddingも幅に含まれる。840pxと指定すると、中身は約808pxになってしまうため、paddingの分を足している
- `box-sizing: content-box;`だと、paddingは幅に含まれない。この場合にpaddingを足すと、デザインより広くなる
- ZeroPlusの教材は`border-box`で統一している。案件や実務では、reset.cssに何が書かれているか必ず確認する

ホバーアニメーション

PCサイズ時のリンクなどにホバーした時の変化をわかりやすくする工夫。`transition`を使うと状態の変化を滑らかにできる（第2回参照）。

```
.sample_link {
  padding-top: 4px;
  border-bottom: 4px solid transparent;
  transition: border-bottom-color 0.24s;
}

.sample_link:hover {
  border-bottom-color: #fff;
}
```

- 最初から`border-bottom: 4px solid transparent;`（透明な線）を引いておき、ホバー時に色だけを変えることで、線の太さ分レイアウトがガタつくのを防いでいる
- 線を背景と同じ色にするより`transparent`の方がよい。背景色が変わったとき、背景と同じ色だと2か所直す必要があるが、透明なら1か所で済む
- 👣 背景色を透明にする（transparent）
- 👣 ホバーした時のスタイルの書き方（疑似クラス）

記事にホバーしたらサムネイルを拡大する

```
@media screen and (min-width: 1080px) {
  .sample_thumb {
    transition: transform 0.36s;
  }

  .sample_post:hover .sample_thumb {
    transform: scale(1.06);
  }
}
```

- `.sample_post:hover .sample_thumb`のように書くと、「記事全体にホバーしたとき、その中のサムネイルだけを変化させる」ことができる
- 画像を囲っている要素（`.sample_thumb-wrapper`など）に`overflow: hidden;`をかけておくと、拡大した画像が枠からはみ出さない（第1回で`img`を`div`で囲った理由の1つ）
- ホバーはマウス操作が前提なので、PCサイズのメディアクエリ内にだけ書いている

画面幅で背景画像を出し分ける

```
@media screen and (min-width: 768px) {
  .top_kv {
    background-image: url("../img/img_kv_lg.jpg"); /* PC用の画像に切り替える */
  }
}
```

- データ量を減らしたり、画質を保ったりするために、スマホとPCで別の画像を使う

「>」で直下の要素だけを指定する

```
/* .posts__col-4 の直下の .post だけ：2列 */
@media screen and (min-width: 768px) {
```

```
  .posts__col-4 > .post {
    width: calc((100% - 24px) / 2);
  }
}

/* .posts__col-3 の直下の .post だけ：3列 */
@media screen and (min-width: 768px) {
  .posts__col-3 > .post {
    width: calc((100% - 48px) / 3);
  }
}
```

- `>`は「直下の」という意味（子セレクタ）
- 同じ`.post`でも、どのセクションの中にあるかでスタイルを変えられる。`>`がないと全部の記事に同じスタイルが当たる
- 記事1個分の幅＝（全体 − 余白の幅 × 余白の個数）÷ 列数（ calc()関数の使い方）

割合でコンテンツ幅を決める

- 例：メインが728px、サイドバーが312px、全体が1080pxのとき、それぞれ約67%・約28%なので、おおまかに`70%`・`30%`にする
- `max-width`は指定の幅より小さい画面には影響しないので、メディアクエリで区切らずベースのスタイルに書いてよい
- max-widthとmin-widthの使い方

中間幅も崩れないようにする

- デザインにないタブレット幅でも崩れないように、検証ツールで見ながら目分量でブレイクポイントを足すこともある（例：580pxでナビを中央寄せ）
- うまく対応することを「よしなに対応する」と言う

特定の画面幅だけ改行を制御する

`br`タグで改行を操作している場合、`br`タグに`display: none;`をかけることで、その画面幅では改行しないようにできる。レスポンシブ時に、特定の要素を非表示にする方法としても使える。

```
@media screen and (min-width: 768px) {
  .sample_break {
    display: none;
```

```
  }
}
```

差分だけを上書きする

レスポンシブ対応は、変化している箇所のみをメディアクエリ内に書けばよい。スタイル全体を書き直す必要はなく、最大幅（`max-width`）や文字サイズなど、その画面幅で変わる部分だけを追加・上書きする。

headタグ内を考える

head内には、サイトごとに書き換えが必要な情報（ファビコン・Webクリップアイコン・OGPなど）をまとめて記述する。

ファビコン・Webクリップアイコン

- **ファビコン**：ブラウザのタブに表示される小さいアイコン
- **Webクリップアイコン**：スマートフォンでサイトをホーム画面に追加した時に表示されるアイコン
- ファビコンを複数の形式（ico・png・svg）で設定するのは、いろいろなデバイスやブラウザに対応するため。ブラウザによって対応する形式が違い、形式ごとに特徴もある（svgは高解像度の画面に強いなど）

```
<!-- favicon/web-clip-icon -->
<link rel="icon" href="favicon.ico" type="image/ico" />
<link rel="icon" href="favicon.png" type="image/png" />
<link rel="icon" href="favicon.svg" type="image/svg+xml" />
<link rel="apple-touch-icon" href="webclip.png" />
```

- 形式ごとの書き方は「HTML ファビコン png」などで調べる
-  ファビコンの作成と設定について

OGP（Open Graph Protocol）

SNSなどでURLがシェアされた時に表示される、タイトル・説明文・画像などを指定する仕組み。

模範コード

```
<!-- favicon/web-clip-icon -->
<link rel="icon" href="favicon.ico" />
<link rel="icon" href="favicon.svg" type="image/svg+xml" />
<link rel="apple-touch-icon" href="webclip.png" />
```

```
<!-- ogp -->
<meta property="og:site_name" content="ZeroPlus Blog" />
<meta property="og:url" content="URL" />
<meta property="og:type" content="article" />
<meta property="og:title" content="ZeroPlus Blog" />
<meta property="og:description" content="サイトの説明文" />
<meta property="og:image" content="シェア時に表示する画像のURL" />
<meta property="og:locale" content="ja_JP" />
<meta name="twitter:card" content="summary_large_image" />
<meta name="twitter:description" content="サイトの説明文" />
<meta name="twitter:image:src" content="シェア時に表示する画像のURL" />
```

これを`<meta name="format-detection" content="telephone=no" />`と`<!-- google fonts -->`の間に入れる。

解説

- metaタグ内に必要な情報を書くと、より（SEOやSNSシェアの観点で）強いサイトになる
- head内の記述は覚える必要はなく、都度コピペして値だけ書き換えればよい
- OGPは、サーバーにアップしたサイトなら OGPチェッカーで確認できる。ローカル環境で確認したいときは localhost OGP チェッカーを使う

【卒業後に学ぼう】

- CSS Gridの使い方：グリッドレイアウト

今日の授業のポイント

1. レスポンシブに必要な材料は3つ：ビューポート／メディアクエリ／ブレイクポイント
2. レスポンシブの作業の流れ：デザインと見比べる→違いを見つける→CSSで書いていく
3. メディアクエリを書く場所は、もともとのCSSのすぐ下！
4. `margin`の左右の値を`auto`にすると中央寄せができる
5. `flex`の子要素が潰れた場合は、潰れてほしくない要素に`flex-shrink: 0;`をつける
6. `display: flex;`は横並びにしたい要素の直前の親要素に当てる

参考記事

- 必要なmetaタグを押さえよう

- 様々なデバイスに対応するためにレスポンシブを学ぼう
- デザインデータからコーディング！ブログサイトのレスポンシブ化
- flexboxを使いこなそう
- viewportの詳細（Qiita）
- レスポンシブのやり方と画面幅の変え方
- flexboxで横幅を縮まらないようにする
- 背景色を透明にする（transparent）
- CSSのtransitionプロパティでアニメーションをかけよう
- CSSの疑似クラス